

PROGRAMMING DECAY

Skill Module Codebook

A Multi-Scalar Design Intelligence Framework for
Post-Industrial Bioremediation at Dagenham Dock, East London

Cluster	RC18
Student Number	25141513
Programme	UCL Bartlett B-Pro MArch Urban Design
Academic Year	2025-2026

GIS	Dimitra Bra
Generative AI	William Huang
Unreal Engine	Enriqueta Llabres-Valls

[GitHub Repository](#)

OneDrive (Large Files): *RC18_25141513_CodebookLargeFiles*

Table of Contents

1. Repository Overview
2. Module 01: Relational Cartographies (GIS)
 - 2.1 Feature Extraction (ArcPy)
 - 2.2 Composite Contamination Index
3. Module 02: Generative AI
 - 3.1 System 01: K-Means Clustering
 - 3.2 System 02: N8N Automation / Entropy Atlas
 - 3.3 System 03: Latent Walk
 - 3.4 System 04: Pix2Pix (Conditional GAN)
 - 3.5 System 05: Neural Cellular Automata
 - 3.6 Pix2Pix to NCA Bridge Pipeline
4. Module 03: Unreal Engine
5. Video Links
6. References

1. Repository Overview

All code and data are hosted at: https://github.com/jiajiajiang91-design/RC18_25141513_SkillModule

```
RC18_25141513_SkillModule/
|-- 01_GIS/
|   |-- scripts/      (ArcPy notebooks)
|   |-- data/         (Grid_Features.csv, sentiment CSV)
|   +-- outputs/      (64 cell map tiles)
|-- 02_AI/
|   |-- 01_KMeans/     (kmeans_grid.py, results)
|   |-- 02_N8N/        (workflow JSONs, entropy_atlas.html)
|   |-- 03_LatentWalk/ (ComfyUI workflow, scripts)
|   |-- 04_Pix2Pix/    (train.py, inference.py, config.py)
|   +-- 05_NeuralCA/   (decay_neural_ca.py, pix2pix_to_ca.py)
+-- 03_UE/
    +-- README.md      (OneDrive link to UE project)
```

2. Module 01: Relational Cartographies (GIS)

Tutor: Dimitra Bra

[01_GIS/ on GitHub](#)

2.1 Feature Extraction (ArcPy)

This script extracts six environmental features from a fishnet grid (8x8 = 64 cells) overlaid on the London Borough of Barking & Dagenham in ArcGIS Pro. Each cell covers approximately 690m x 525m. Features: Brownfield density, Building coverage, Waste infrastructure, Flood risk exposure, Population density, and Ecological sensitivity.

[01_feature_extraction.ipynb](#)

```
import arcpy, csv

data = {}

# F1: Brownfield site count per cell
with arcpy.da.SearchCursor("F1_result", ["Cell_ID", "F1_Brownfield"]) as cur:
    for row in cur:
        data[row[0]] = {"F1": row[1] if row[1] else 0}

# F2: Building footprint coverage (%)
with arcpy.da.SearchCursor("F2_result",
    ["Cell_ID", "sum_Area_SQUAREMETERS", "Shape_Area"]) as cur:
    for row in cur:
        cid, val, sa = row[0], (row[1] if row[1] else 0), row[2]
        if cid not in data: data[cid] = {}
        data[cid]["F2"] = val / sa * 100 if sa else 0

# F3-F6: Waste, Flood, Population, Ecological Sensitivity
# (same pattern – spatial join results queried per cell)
# ... see full script on GitHub

# Output: Grid_Features.csv (64 rows x 7 columns)
```

2.2 Composite Contamination Index

Min-max normalisation (0-100) across all six features, then arithmetic mean to produce a single Composite Index per cell. The highest-scoring cells (37, 50, 36, 26) correspond to Dagenham Dock — validating site selection.

[02_composite_index.ipynb](#)

```
import arcpy, numpy as np

# Min-max normalisation (0-100) per feature
normed = np.zeros_like(arr)
for i in range(6):
    col = arr[:, i]
    mn, mx = col.min(), col.max()
    normed[:, i] = (col - mn) / (mx - mn) * 100 if mx > mn else 0

# Composite = arithmetic mean of all normalised features
composite = normed.mean(axis=1)
# Write back to ArcGIS Pro attribute table
```

3. Module 02: Generative AI

Tutor: William Huang

[02_AI/ on GitHub](#)

Five-system pipeline: K-Means (classify) -> N8N (map entropy) -> Latent Walk (imagine) -> Pix2Pix (translate) -> Neural CA (simulate).

3.1 System 01: K-Means Clustering

[kmeans_grid.py](#)

Input: 64 cells x 6 features (Grid_Features.csv). Method: K-Means with K=6, validated by Elbow Method + Silhouette Score + PCA projection. Output: Cluster 0 = 7 cells = Dagenham Dock priority intervention zone. Libraries: scikit-learn, pandas, matplotlib.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA

# Normalise features to 0-1 range
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_raw)

# Elbow method to find optimal K
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    km.fit(X_scaled)

# Final clustering with K=6
km_final = KMeans(n_clusters=6, random_state=42, n_init=50)
df["Cluster"] = km_final.fit_predict(X_scaled)

# PCA for 2D visualisation
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# PC1: 45.9% variance, PC2: 29.3% variance
```

3.2 System 02: N8N Automation / Entropy Atlas

[02_N8N/ on GitHub](#)

Two N8N workflows automate data collection: (1) Overpass API extracts 986 OSM sites across Dagenham-Barking-Rainham; (2) Google Places API scrapes 40 sentiment reviews. A coordinate-based naming script resolved 670 unnamed entries. Output: 1,026 geo-located data points visualised as an interactive Leaflet map (Entropy Atlas).

[Interactive Map: Entropy Atlas](#)

[Video: N8N Entropy Atlas Demo](#)

Tech stack: Leaflet 1.9.4, MarkerCluster, PapaParse, CartoDB Dark Matter basemap. CSV data embedded in HTML script tag for self-contained deployment.

3.3 System 03: Latent Walk

[03_LatentWalk/ on GitHub](#)

1,208 images of industrial ruins and rewilding vegetation mapped into 3D latent space via PCA + t-SNE embedding. Stable Diffusion interpolation in ComfyUI generates 90-frame material futures matrix across three trajectories: Concrete x DECAY, Brick x REWILD, Steel x REUSE.

Video: [Latent Walk Video](#)

ComfyUI Workflow: [latent_walk_60frames.json](#)

3.4 System 04: Pix2Pix (Conditional GAN)

[04_Pix2Pix/ on GitHub](#)

Training data: 60 floor plans (30 Experience Centres + 30 Recycling Factories), abstracted into paired datasets: Domain A (linear outline) to Domain B (functional colour zoning). Architecture: Pix2Pix (Isola et al., 2017). Hyperparameters: Epochs=480, Lambda L1=100, LR=0.0002, Batch=1, Image size=256.

Key scripts:

[train.py](#) -- [inference.py](#) -- [config.py](#)

```
# config.py - key hyperparameters
IMG_SIZE      = 256
EPOCHS        = 480
LR             = 0.0002
LAMBDA_L1     = 100      # L1 loss weight
BATCH_SIZE    = 1

# Colour mapping: functional zones
OUTPUT_CLASSES = {
    "RED":      (255, 0, 0),      # Processing Hall / Exhibition Gallery
    "GREEN":    (0, 255, 0),      # Office
    "BLUE":     (0, 0, 255),      # Storage / Impact Simulation
    "CYAN":     (0, 255, 255),    # Courtyard / Interactive Lab
    "MAGENTA":  (255, 0, 255),    # Leisure Area
}
```

Post-processing: Delaunay triangulation rationalises diffuse pixel fields into vectorised partitions. Three stable epochs (220, 360, 420) identified via interval tracking; their intersecting red zones form a spatial consensus for the Ford Stamping Plant layout.

3.5 System 05: Neural Cellular Automata

[05_NeuralCA/ on GitHub](#)

Based on Mordvintsev et al. (2020) 'Growing Neural Cellular Automata'. The NCA learns from a four-colour remediation strategy map: DECAY (19.6%), REWILD (15%), REUSE (65.4%), DISMANTLE (7.46%). Training: 16 channels, 16,000 steps, 3x3 local perception. Key insight: identical rules + different initial noise produce varied but valid outcomes (Simulation as Inquiry, cf. Batty 2005).

[decay_neural_ca.py](#)

3.6 Pix2Pix to NCA Bridge Pipeline

[pix2pix_to_ca.py](#)

Three-stage bridge: (1) K-Means colour quantisation converts Pix2Pix continuous gradients into discrete colour blocks; (2) Euclidean distance mapping assigns each cluster to the nearest strategy colour (DECAY/REWILD/REUSE/DISMANTLE); (3) 16-channel CA initial state matrix built with noise injection and dropout matching training parameters.

```
# Strategy colour mapping (Dagenham 4-colour)
_DECAY      = (0.886, 0.294, 0.290) # Zone C industrial core
_REUSE      = (0.216, 0.541, 0.866) # Zone B warehouse reuse
_DISMANTLE  = (0.937, 0.624, 0.153) # Zone D Ford plant
_REWILD     = (0.388, 0.600, 0.133) # Zone E+F wetland

# Pix2Pix colour -> NCA strategy mapping
# red      -> DECAY      (heavy industry / high concrete)
# green    -> REWILD     (organic / green corridor)
# blue     -> REUSE      (warehouse / material reuse)
# cyan     -> REWILD     (open courtyard)
# magenta  -> DISMANTLE  (low density / dismantlable)
```

```
# Pipeline: Pix2Pix output -&gt; K-Means quantise -&gt; classify
#           -&gt; build 16-ch state (64x64) -&gt; pad to 96x96
```


4. Module 03: Unreal Engine

Tutor: Enriqueta Llabres-Valls

[03_UE/ on GitHub](#)

[Video: UE Niagara Systems Demo](#)

Niagara System 1: Industrial Smoke Plume

SimpleSpriteBurst emitter. Lifetime: 3-5s. Modules: Initialize Particle (Sprite 1000), System Location (vertical column), Add Velocity Linear (upward drift), Scale Color (white to grey curve), Drag (2.0), Scale Sprite Size (expanding cloud). Material: T_Smoke_SubUV_01_Mat (Translucent, SubUV flipbook, Particle Color x Texture to Emissive, 250 instructions / 8 resources).

Niagara System 2: Spark / Ember Burst

SimpleSpriteBurst, 500 particles. Lifetime: 0.5-3s. Modules: Shape Location Sphere (R=5), Add Velocity In Cone (300-500, 45 deg), Color from Curve (white to orange to red to black), Gravity Force, Curl Noise Force (Strength 10, Freq 50), Collision (Ray Traced, Restitution 0.6). Material: flame02_Mat (Translucent, Particle Color x 100 to Emissive for HDR bloom, alpha-masked flame atlas, 253 instructions / 8 resources).

UE project files (>100MB) hosted on OneDrive: RC18_25141513_CodebookLargeFiles

5. Video Links

Module	Description	URL
Generative AI	Latent Walk: Decay Trajectories	https://youtu.be/XniW9sVhLH0
Generative AI	N8N Entropy Atlas Demo	https://youtube.com/watch?v=XA8W4Zhc5Ek
Unreal Engine	Niagara Smoke + Spark Systems	https://youtu.be/oyOwx5gqYzE

[Interactive Map: Entropy Atlas \(live\)](#)

6. References

- [1] Batty, M. (2005). Cities and Complexity. MIT Press.
- [2] Isola, P. et al. (2017). 'Image-to-Image Translation with Conditional Adversarial Networks.' CVPR.
- [3] Mordvintsev, A. et al. (2020). 'Growing Neural Cellular Automata.' Distill.
- [4] Rombach, R. et al. (2022). 'High-Resolution Image Synthesis with Latent Diffusion Models.' CVPR.
- [5] Gilpin, W. (2019). 'Cellular Automata as Convolutional Neural Networks.' Physical Review E.
- [6] Pedregosa, F. et al. (2011). 'Scikit-learn: Machine Learning in Python.' JMLR.
- [7] Huang, W. (2025-2026). RC18 Generative AI Module. UCL Bartlett B-Pro.

Software: ArcGIS Pro 3.6, Python 3.10+, PyTorch 2.0+, scikit-learn, ComfyUI, N8N, Unreal Engine 5.4, Leaflet.js